AIDE — System Report

Adaptive Intelligent Dynamic Embedding Framework

What the project does • what runs in the backend after an image is uploaded • what we are discovering

P. Yogishwar • 23261A6746 • B.Tech CSE (Data Science), MGIT Hyderabad • CD753PC Project Stage I

Repository: github.com/yogishwar09/aide-steganography

Metric	Measured value
Decision-engine training set	334 real BOSSbase images, ~342,000 image blocks
Method-classifier validation accuracy	0.877 (bpp regressor MAE ~0.14)
Embed time (CPU, ~0.5 MP image)	~360 ms | Recover time ~30 ms
Memory footprint (RSS delta per embed)	~12–13 MB
Stego image quality (no attack)	PSNR ~33–51 dB, SSIM ~0.92–0.98
Automated tests	74 passing (1 boundary-sensitive xfail, documented)
Hardware requirement	CPU only — no GPU / CUDA anywhere

All numbers in this document are measured from the running system and locked by an automated test — nothing here is an aspiration or an estimate.

1. What the project does

AIDE is a CPU-only image steganography system for defence and intelligence settings. It covers **both sides** of covert information transfer in a single tool:

1.1 Operator side — secure covert communication

Hide an encrypted message inside an ordinary-looking photograph so that interception reveals nothing, and so that the message can still be recovered after the image has been compressed, filtered, noised, cropped, or partially replaced in transit. This is the sender/receiver role: satellite & drone imagery relay, military secure communication, government document exchange.

1.2 Analyst side — blind threat detection (steganalysis)

Take an **arbitrary intercepted image** — no shared secret, no side-channel data, no assumption about whether or how it was modified — and decide whether it is likely carrying hidden data. This is the counter-intelligence role: border and port media screening, cyber-defence command-centre monitoring for covert data exfiltration.

1.3 How you use it

- **Command line:** `aide embed / extract / attack / evaluate / detect / train`
- **Local web console** (`python webapp/app.py` → browser at 127.0.0.1:5050): an Operator tab and an Analyst tab, click-through, no internet needed.
- **Python API:** `aide.pipeline.embed() / recover()`, `aide.steganalysis.analyze()`.

The novelty is the **AIDE decision engine**: a lightweight trained model that decides, *per 8×8 or 16×16 image block*, four things at once — *where* to hide data, *how many* bits, *which* algorithm (LSB / DCT / DWT), and *whether* the block is reliable enough to use at all. No two images get the same embedding map.

2. What runs in the backend after you upload an image

Below is the exact sequence the web console / CLI executes. Each step names the module that does the work, so this doubles as a code map.

2.1 EMBED path (Operator → "Embed message")

#	Step	Module	What actually happens
1	Load & plane split	pipeline.load_image	Read the uploaded file's raw bytes. If colour, convert RGB→YCbCr and take only the Y (luma) plane; Cb/Cr are kept untouched for later recombination.
2	Block partition	features.partition_blocks	Split the plane into non-overlapping 8×8 blocks (16×16 for images ≥512 px). Each block gets a stable index used everywhere downstream.
3	Per-block features	features.extract_image_features	Four normalised signals per block: texture (Local Binary Pattern + GLCM energy), Shannon entropy, pixel variance, edge strength (Sobel + Canny).
4	AIDE decision engine	model.AIDEModel.decide	Trained decision tree scores each block: $S(b)=\alpha\cdot$texture + $\beta\cdot$entropy + $\gamma\cdot$variance + $\delta\cdot$edge. Blocks below threshold τ are skipped; the rest are ranked and assigned a method (LSB/DCT/DWT) and a bit-rate.
5	Encrypt	crypto.encrypt	AES-256-GCM. Key = SHA-256(image-hash + shared-secret), so the same message in two different photos produces unrelated ciphertext. A GCM authentication tag is attached.
6	Error-correction code	ecc.encode	Reed-Solomon RS(255,223): every 223 payload bytes get 32 parity bytes → up to 16 corrupted bytes (or 32 known-erased bytes) recoverable per 255-byte codeword. ~14% overhead.
7	Interleave	ecc.interleave	Scatter each codeword's bytes across many non-adjacent blocks, so a localised loss (a crop, a splice) damages a few bytes of many codewords instead of destroying whole codewords.
8	Adaptive embedding	embedding.lsb / dct / dwt	Write the interleaved bits into each block using its assigned method: LSB substitution for flat regions; mid-frequency DCT coefficient-pair QIM for textured regions; 2-level Haar DWT coarse-sub-band QIM for edge regions.
9	Tamper references	tamper.build_references	For every block actually used, store a SHA-256 hash and an 8-bin histogram in a sidecar JSON, for reference-based tamper localisation at recovery time.
10	Output	pipeline / webapp	Return the stego image (colour recombined if needed) plus the sidecar JSON, and report PSNR, SSIM, blocks used, and the LSB/DCT/DWT method mix.

2.2 RECOVER path (Operator → "Recover")

The reverse of steps 8→5, driven entirely by the sidecar (no ML model needed at the receiver): recompute each block's hash and compare to the reference → a spatial *tamper map*; extract bits per the stored method map; reverse the interleave; Reed-Solomon decode, feeding the tamper-flagged blocks in as *known erasures* (doubling correction power); AES-256-GCM decrypt and verify the tag. Tag failure = "tampered beyond recovery".

2.3 DETECT path (Analyst → "Analyze")

Completely independent of the sidecar. Takes only the image:

- **Chi-square pairs-of-values test** (Westfeld & Pfitzmann, 2001) — the primary signal. LSB embedding of an encrypted (near-random) payload equalises the counts of value pairs (2k, 2k+1); natural photos do not have this. Run on the whole image and per horizontal row-band.
- **LSB / next-bit-plane correlation** — a real photo's least-significant bit plane is weakly correlated with its finer structure; overwriting it with random payload collapses that correlation toward zero.
- **LSB bit-balance** — deviation of the LSB plane from a 50/50 split (weakest signal, low weight).
- Combined into a suspicion score and a verdict: *clean* / *suspicious* / *likely steganographic*.

3. What we are discovering (measured findings)

The project's method is “measure, don't assume”: every claim below is backed by a test on real BOSSbase images, and several of the original report's assumptions did **not** survive contact with measurement.

3.1 What reliably works

Scenario	Result	Note
No attack	100% exact recovery	PSNR clears the report's 42 dB target for short messages on textured covers.
JPEG compression, quality 30–70	Recovers reliably	Meets and exceeds report objective O4's stated target of Q=50.
Localised splicing / region replacement	Recovers	When the message spans several RS codewords so interleaving can spread the damage. Tamper localisation flags 100% of genuinely replaced blocks.
Mild Gaussian noise (σ≈10), Gaussian blur	Recovers reliably	—
3×3 median filter, mild (0.75×) scaling	Recovers	Only after the DWT and DCT fixes described in 3.2–3.3.
Wrong shared secret or tampered ciphertext	Always rejected	AES-GCM tag verification never lets a corrupted message through as valid.
Colour (RGB) covers	Full colour fidelity	Embed in luma only; chroma passes through untouched; survives a real PNG save/reload + attack.

3.2 Finding: the DWT method as originally conceived did not work

Single-level Haar DWT embedding in the horizontal/vertical *detail* sub-bands measured **~0% bit survival** against median filtering and geometric scaling — at *any* coefficient separation tested, up to delta = 400 (well past visible distortion). Both attacks resample the exact 2×2 neighbourhood a single-level Haar transform depends on. This directly contradicts the report's assumption that DWT gives “superior robustness against geometric attacks”. **Fix:** embed in the *coarse* 2-level approximation sub-band instead (a twice-averaged, low-frequency view of the block). Measured result: 100% survival against JPEG Q30/50, median filtering, and 2× downscale-then-restore, at delta = 40 and a *better* per-block PSNR (~37 dB vs ~29 dB) than the broken version needed.

3.3 Finding: DCT has a hard capacity–versus–robustness trade-off

The more DCT capacity a block is asked for, the further into higher, less-robust mid-frequencies it must reach. Measured on 16×16 blocks against a 3×3 median filter: 13 coefficient pairs → 20/20 blocks survive; 26 pairs → 10/20; 39+ pairs → 0/20 — and raising the coefficient separation does **not** help, because it is about *which* frequencies are used, not how far apart the pair is pushed. **Fix:** cap DCT to a measured-safe frequency band. Cost: less capacity per block, so more blocks are needed — which surfaces as an honest “capacity insufficient” error rather than a silent recovery failure.

3.4 Finding: LSB is fragile to essentially everything

Measured ~45–55% bit-error rate under every attack tried, including a plain re-save. LSB depends on exact pixel values. Its error rate is so much higher than DCT/DWT's that even 1% of the payload coming from LSB was measured to flip an otherwise-passing recovery into a failure. **Fix:** the payload allocator caps LSB at 0.5% and DWT at 2% of the RS-encoded payload; DCT (the most broadly robust method after 3.3) carries the rest. LSB is kept non-zero only so the “adaptive, multi-method” behaviour stays visible.

3.5 What still does not work (documented, not hidden)

- **Aggressive 0.5× scaling** — and the report itself calls scaling attacks “the most damaging”. Per-codeword error counts land far past the correction budget even with DCT carrying ~97% of the payload.
- **Gaussian noise** $\sigma \geq 20$ (a report O4 target) and **salt-and-pepper noise** — this one is a deliberate dial: raising DCT's coefficient separation from 40 to ~150 makes σ=20 fully recoverable, at a real image-quality cost (block PSNR ~30 dB $\rightarrow$ ~20 dB). Shipped conservative by default.
- **Cropping / splicing beyond ~2–3%** of the image relative to how many codewords the message spans — exceeds the erasure budget outright, though detection still works.
- **Single-codeword (very short) messages** get zero benefit from interleaving — with one codeword there is nothing to spread damage *across*.

3.6 Finding: the detection side, and what it says about AIDE's own stealth

Test	Measured outcome
Clean real images falsely flagged	1 in 40 (2.5%) — consistent with the statistical test's own significance level, not a bias.
Naive full-image sequential LSB steganography	Detected in 100% of cases; detection degrades honestly at lower embedding rates (a known limit of the classical whole-image test).
AIDE's *own* adaptive stego images	0 of 15 flagged — a verified answer to the report's own “Detectability (RS analysis): Negative” claim, *scoped to this classical detector only*.

Why AIDE evades this detector: its LSB share is capped under 1% (too little to shift the whole-image pair-equalisation statistic), and the bulk of the payload lives in DCT/DWT coefficients that a spatial-LSB-only detector cannot see. This is a **scoped** claim — it says nothing about more advanced steganalysis (Sample Pairs Analysis, RS rate estimation, transform-domain or ML-based detectors), which are documented as not implemented.

3.7 A finding about method itself

A false-positive regression in the detector was caught *by testing* before shipping: an early scoring change that folded a per-row-band breakdown into the verdict amplified 1/40 clean images being flagged into 16/20. It was reverted, and the row-band data kept as a labelled “diagnostic only” display. The general pattern — a plausible improvement that measurement rejects — recurred often enough to be the project's main working lesson.

4. System facts

4.1 The trained decision engine

Training is empirical, not circular: for each block, method labels come from the report's own texture/edge rule (with thresholds set from data percentiles), and the bit-rate label comes from actually embedding random bits at each level and *measuring* the resulting block PSNR. A DecisionTreeClassifier learns method; a DecisionTreeRegressor learns bit-rate; the classifier's normalised feature importances become the explainable weights ($\alpha,\beta,\gamma,\delta$). Measured learned weighting: texture ≈ 0.55, edge ≈ 0.44, entropy and variance ≈ 0.01 combined — i.e. the model relies almost entirely on texture and edge content.

4.2 Cryptography & coding parameters

Component	Parameter
Cipher	AES-256 in GCM mode (FIPS 197), 12-byte nonce, 16-byte tag
Key derivation	SHA-256(SHA-256(cover file bytes) + shared secret)
Error-correcting code	Reed-Solomon RS(255,223), t = 16 (or 32 known erasures) per codeword
Interleaving	Column-major over the codeword matrix — adjacent embedded bytes belong to different codewords
DCT embedding	Mid-frequency zigzag coefficient pairs, relative-magnitude QIM, separation delta = 40, frequency-band capped for robustness
DWT embedding	2-level Haar, coefficient pairs in the coarse approximation sub-band, separation delta = 40
Payload allocation caps	LSB ≤ 0.5%, DWT ≤ 2% of the RS-encoded payload; DCT unbounded

4.3 Attack simulation library

Eight concrete simulations across the report's six categories: JPEG compression, Gaussian noise, salt-and-pepper noise, cropping (fixed-canvas region removal), splicing, median filter, Gaussian filter, and geometric scaling (downscale-then-restore). All operate on the luma plane and preserve image dimensions so block alignment for recovery stays valid.

4.4 Technology stack

Python 3, CPU only. NumPy, OpenCV, scikit-learn, scikit-image, SciPy, PyWavelets, PyCryptodome, reedsolo, Pillow for the core; Flask for the web console; pytest for the 74-test suite. No GPU, CUDA, or proprietary hardware anywhere — deployable on a Raspberry Pi 4 / Jetson-class board, as the report intends.

Generated from the running AIDE system on 2026-09-10. Full source, tests, and the honest “what works / what doesn't” breakdown: github.com/yogishwar09/aide-steganography (see README.md).